

JavaAgent CLI 伪代码文档

本文档用自然语言描述整体架构和核心流程，不包含任何可运行代码。

包结构总览

com.javagent

- |— JavaAgentCLI # 程序入口
- |— core/ # 核心逻辑
 - | |— Agent # Agent 引擎
 - | |— Config # 配置管理
 - | |— ConversationManager # 对话管理
 - | |— ApprovalManager # 审批管理
 - | |— ApprovalHandler # 审批回调接口
 - | |— ApprovalDecision # 审批决策枚举
 - | |— ApprovalOutcome # 审批结果
- |— model/ # 数据模型
 - | |— Role # 角色枚举
 - | |— Message # 消息记录
 - | |— ToolCall # 工具调用请求
 - | |— ToolResultMessage # 工具结果消息
 - | |— ModelResponse # 模型响应
 - | |— ModelClient # 模型客户端接口
 - | |— MockModelClient # 模拟客户端
 - | |— OpenAiCompatibleModelClient # OpenAI 客户端
 - | |— TextStreamHandler # 流式输出接口
- |— tools/ # 工具集
 - |— Tool # 工具接口
 - |— ToolDefinition # 工具定义
 - |— ToolExecutionResult # 工具执行结果
 - |— ToolRegistry # 工具注册中心
 - |— FileToolSupport # 文件工具辅助
 - |— ReadFileTool # 读文件
 - |— GrepTool # 文本搜索
 - |— ListDirectoryTool # 目录列表
 - |— WriteFileTool # 写文件
 - |— DeleteFileTool # 删除文件
 - |— BashTool # Shell 命令

一、com.javagent 包

JavaAgentCLI —— 程序入口

作用

核心字段

- 配置对象、对话管理器、工具注册中心、模型客户端、Agent 引擎

方法流程

main

- 创建 JavaAgentCLI 实例，调用 run 方法

run

- 加载默认配置（从 config.properties 文件）
- 解析命令行参数（--mock、--real、--api-key、--help）
- 初始化/重建运行时组件
- 启动命令行交互循环

applyArgs

- 遍历命令行参数数组
- 遇到 --mock 则切换为模拟模式
- 遇到 --real 则切换为真实模式
- 遇到 --api-key 则设置 API 密钥
- 遇到 --help 则打印帮助并退出

rebuildRuntime

- 如果对话管理器为空则创建新实例（已存在则复用）
- 创建工具注册中心，依次注册：读文件、搜索、目录列表、写文件、删除文件
- 如果 Bash 已启用，注册 Bash 工具
- 根据当前模式选择模型客户端（模拟模式用 Mock，真实模式用 OpenAI 兼容客户端）
- 创建 Agent 实例，传入配置、模型客户端、工具注册中心、对话管理器

startCli

- 打印启动横幅
- 显示当前模式
- 尝试恢复上次的会话
- 进入主循环：
- 显示提示符 "javaagent> "
- 读取用户输入
- 如果输入为空则跳过
- 如果以 / 开头则处理命令
- 否则交给 Agent 处理，传入审批回调和流式输出处理器
- 根据流式输出是否已触发，决定直接打印还是换行

handleCommand

- 解析命令和参数
- /help → 打印帮助
- /exit 或 /quit → 退出程序
- /clear 或 /new → 开始新会话
- /save → 保存当前会话
- /load → 加载指定会话（按 ID 或标题）

- /sessions → 列出所有保存的会话
- /tools → 列出可用工具
- /mode → 切换模式 (mock/real) , 切换后重建运行时
- /stream → 开关流式输出
- /bash → 开关 Bash 工具, 切换后重建运行时
- /prompt → 查看/设置/重置自定义系统提示词
- /approvals → 查看审批缓存条目数或清空缓存
- /status → 显示当前状态信息

promptApproval

- 打印工具调用信息 (工具名、参数)
- 提示用户输入 yes/no/cancel
- 根据用户输入返回对应的审批决策

ConsoleTextStreamHandler (内部类)

- 实现 TextStreamHandler 接口
- 每收到一块文本就立即打印到控制台
- 记录是否已输出过文本, 用于判断是否需要额外换行

二、com.javagent.core 包

Agent —— 核心引擎

作用

核心字段

- 配置、模型客户端、工具注册中心、对话管理器、审批管理器

processTurn (无流式)

- 调用带流式参数的版本, 传入 null

processTurn (核心方法)

- 把用户消息加入对话历史
- 构建系统提示词 (告诉 AI 它是谁、有哪些工具可用)
- 进入 Agent Loop (最多循环 maxIterations 次) :
- 根据配置决定是否启用流式输出
- 调用模型客户端发送请求, 获取响应
- 如果是错误响应 → 记录错误消息, 自动保存, 返回错误文本
- 如果是纯文本响应 → 记录 AI 回复, 自动保存, 返回回复文本
- 如果是工具调用响应 → 记录 AI 的工具调用消息, 逐个执行工具调用, 将结果加入对话历史, 继续循环
- 超过最大循环次数 → 记录提示消息, 自动保存, 返回提示

executeToolCall

- 从工具注册中心查找工具, 找不到则返回错误
- 通过审批管理器检查是否被授权
- 审批不通过则返回拒绝原因
- 审批通过则执行工具, 返回执行结果

buildSystemPrompt

- 设定 AI 身份 ("JavaAgent CLI, 简洁的编程助手")
- 添加使用原则 (按需使用工具、工作区内操作、避免破坏性操作)
- 如果有自定义提示词则追加
- 列出所有可用工具的名称、描述和必填参数

autoSaveQuietly

- 如果配置了自动保存, 则保存当前会话
- 保存失败时静默忽略, 不影响主流程

Config —— 配置管理器

作用

配置项

配置项

说明

默认值

agent.mock_mode

是否使用模拟模式

true

agent.api_key

API 密钥

空

agent.base_url

API 基础 URL

https://api.openai.com/v1

agent.model

模型名称

gpt-5.4-mini

agent.auto_save

是否自动保存会话

true

agent.max_iterations

最大工具调用循环次数

6

agent.enable_bash

是否启用 Bash 工具

false

agent.stream_responses

是否流式输出

true

agent.system_prompt

自定义系统提示词

空

agent.approval_cache

是否启用审批缓存

true

agent.allow_external_paths

是否允许访问工作区外路径

false

方法流程

loadDefault

- 工作目录设为当前目录，应用主目录设为 ~/.javaagent-cli
- 调用内部加载方法

load

- 从文件读取配置 → 填充缺失的默认值 → 保存回文件

配置文件查找顺序

路径方法

- stateDirectory: 状态目录（优先在工作目录下创建 .javaagent-cli，否则放 appHome）
- sessionPath: 旧版会话文件路径
- sessionDirectory: 会话存储目录
- currentSessionMarkerPath: 当前会话标记文件

ConversationManager —— 对话管理器

作用

核心字段

- 配置对象、Jackson 序列化器
- 当前会话 ID、标题、消息列表、开始时间、最后一条 AI 回复

方法流程

构造器

- 保存配置引用
- 初始化 Jackson 序列化器（支持 Java 时间类型、日期用字符串、格式化输出）
- 开始一个新的空会话

addUserMessage / addAssistantMessage / addAssistantToolCallMessage /
addToolResultMessage

- 分别创建对应角色的消息对象，加入当前消息列表

currentContext

- 返回当前消息列表的不可变副本

startNewSession

- 生成新的会话 ID (UUID)
- 设置标题（传入标题或默认标题）
- 清空消息列表
- 重置会话开始时间和最后回复

saveCurrentSession

- 创建会话快照（包含 ID、标题、开始时间、更新时间、消息列表、最后回复）
- 确保目录存在
- 保存到三处：会话目录下的 JSON 文件、旧版路径、当前会话标记文件

loadLastSession

- 优先读取标记文件，获取会话 ID
- 如果找到对应的会话文件则加载
- 否则回退到旧版路径
- 加载后应用快照恢复会话状态

loadSession

- 列出所有保存的会话
- 按 ID 或标题模糊匹配
- 如果是 "latest" 则加载最新的
- 找到则加载快照并更新标记文件

listSessions

- 遍历会话目录下的 JSON 文件
- 提取每个会话的摘要信息
- 按更新时间倒序排列

内嵌数据结构

- SessionSummary: 会话摘要（ID、标题、更新时间、消息数）
- SessionSnapshot: 会话快照（完整会话数据，用于保存/加载）

ApprovalManager —— 审批管理器

作用

核心字段

- 配置对象、审批缓存（Map 结构）

authorize

- 获取工具定义
- 执行策略检查 (evaluatePolicy)
- 如果策略结果为"允许" → 直接返回批准
- 如果策略结果为"拒绝" → 直接返回拒绝
- 如果策略结果为"需审批" → 检查缓存
- 缓存命中 → 沿用缓存的审批决策
- 缓存未命中 → 调用审批回调请求用户确认
- 用户决策不为"取消"时存入缓存
- 返回审批结果

evaluatePolicy (策略检查)

- 如果是 Bash 工具且未启用 → 拒绝
- 提取工具参数中的路径，如果路径在工作区外且不允许外部路径 → 拒绝
- 如果是破坏性工具且路径为受保护路径（.git、javaagent-cli 等） → 拒绝
- 如果是删除工具且目标是工作区根目录或目录 → 拒绝

- 如果工具标记为不需要审批 → 允许
- 其他情况 → 需要用户审批

extractPath

- 从工具参数中提取路径（大多数工具用 "path" 参数，Bash 用 "workingDirectory"）
- 将路径解析为绝对路径

isProtectedPath

- 检查路径是否位于状态目录、配置文件、.git 目录、.javaagent-cli 目录等受保护位置

cacheKey

- 生成缓存键：工具名 + 标准化后的参数（路径参数会解析为绝对路径后排序）

ApprovalDecision —— 审批决策枚举

作用

- APPROVED：同意执行
- DENIED：拒绝执行
- CANCELLED：取消（中止操作）

ApprovalOutcome —— 审批结果

作用

工厂方法

- approved / cachedApproved：创建批准结果（区分是否来自缓存）
- denied / cachedDenied：创建拒绝结果（区分是否来自缓存）

ApprovalHandler —— 审批回调接口

作用

- 唯一方法 request(ToolCall)：接收工具调用请求，返回审批决策
- 标记为函数式接口，可用 Lambda 创建

三、com.javagent.model 包

Role —— 角色枚举

作用

- SYSTEM：系统提示词，告诉 AI 如何行为
- USER：用户输入的消息
- ASSISTANT：AI 的回复
- TOOL：工具执行后返回的结果

每个枚举值持有 apiValue 字段，对应 API 协议中的小写字符串（如 "user"）。

Message —— 消息记录

作用

字段

约束

工厂方法

- system / user / assistant: 创建纯文本消息
- assistantWithToolCalls: 创建包含工具调用的 AI 消息
- toolResult: 创建工具结果消息 (区分成功和失败)

辅助方法

- hasToolCalls: 是否包含工具调用
- isToolError: 工具执行是否出错

ToolCall —— 工具调用请求

作用

字段

工厂方法

ToolResultMessage —— 工具执行结果消息

作用

字段

工厂方法

ModelResponse —— 模型响应

作用

响应类型

- TEXT: 纯文本回复
- TOOL_CALLS: 请求调用工具
- ERROR: 请求出错

工厂方法

辅助方法

ModelClient —— 模型客户端接口

作用

方法

- chat (核心): 发送对话请求, 返回模型响应
- chat (带流式输出): 默认实现, 先获取完整响应, 再将文本分块发送给流式处理器
- name: 返回客户端名称

分块策略

MockModelClient —— 模拟模型客户端

作用

chat

- 如果消息列表为空 → 返回就绪提示
- 如果最后一条是工具结果 → 总结工具执行结果
- 找到最后一条用户消息
- 关键词匹配:
- "删除/delete" → 返回 delete_file 工具调用
- "写入/write" → 返回 write_file 工具调用

- "列出/目录/ls" → 返回 list_directory 工具调用
- "读取/read" → 返回 read_file 工具调用
- "搜索/grep" → 返回 grep 工具调用
- "bash/shell" → 返回 bash 工具调用
- "help/工具" → 返回帮助文本
- 未匹配 → 返回默认文本提示

summarizeToolResult

- 根据工具名称 (read_file、grep、list_directory 等) 生成不同格式的总结
- 截取结果的前若干行

OpenAiCompatibleModelClient —— OpenAI 兼容客户端

作用

chat

- 检查 API Key 是否已配置，未配置则返回错误
- 构建 JSON 请求体
- 发送 HTTP POST 请求到 API 端点
- 检查 HTTP 状态码，400+ 返回错误
- 解析 JSON 响应，转换为 ModelResponse

buildRequestBody

- 设置模型名、温度、工具选择策略
- 构建消息数组：系统提示词 + 对话历史
- 根据消息角色分别处理：USER/SYSTEM 只设内容，ASSISTANT 可能含工具调用，TOOL 需配对工具调用 ID
- 构建工具定义数组：每个工具包含名称、描述、参数 Schema、必填参数列表

parseResponse

- 检查响应中是否有错误对象
- 提取第一个 choice 的 message
- 如果 message 中有 tool_calls 字段 → 解析为工具调用响应
- 否则 → 提取文本内容作为纯文本响应

TextStreamHandler —— 流式输出接口

作用

- 函数式接口，唯一方法 onChunk(chunk)
- 可用 Lambda 创建

四、com.javagent.tools 包

Tool —— 工具接口

作用

方法

- definition：返回工具的元信息定义

- execute: 接收参数 Map，执行操作，返回执行结果

ToolDefinition —— 工具定义

作用

字段

- name: 工具名称
- description: 功能描述
- parameterDescriptions: 每个参数的含义
- parameterTypes: 每个参数的类型
- requiredParameters: 必填参数集合
- requiresApproval: 是否需要用户审批
- readOnly: 是否只读
- destructive: 是否破坏性操作
- aliases: 工具别名列表

ToolExecutionResult —— 工具执行结果

作用

工厂方法

ToolRegistry —— 工具注册中心

作用

核心数据结构

register

- 将工具按名称（小写）存入名称 Map
- 遍历别名，逐个存入别名 Map

find

- 先按名称查找
- 找不到再按别名查找
- 都找不到返回空

definitions

- 返回所有工具的定义列表，用于告诉 AI 有哪些工具

listTools

- 生成格式化的工具列表文本，包含名称、别名、描述、审批标志

FileToolSupport —— 文件工具辅助类

作用

方法

- stringValue: 从 Map 提取字符串值，null 返回空串
- intValue: 从 Map 提取整数值，兼容 Number 和 String 类型
- booleanValue: 从 Map 提取布尔值，兼容 Boolean 和 String 类型
- normalizePath: 规范化路径，去除冗余的 "." 和 ".."
- isBinary: 检测文件是否为二进制文件（通过 null 字节和 UTF-8 解码判断）

ReadFileTool —— 读文件工具

作用

安全标记

安全限制

- 文件大小超过 256KB 拒绝读取
- 二进制文件拒绝读取

execute

- 提取并验证路径参数
- 规范化路径
- 检查文件是否存在且是普通文件
- 检查文件大小和是否为二进制
- 读取文件全部行
- 根据 offset 和 limit 截取指定范围
- 构建带行号的输出格式
- 如果还有更多行，显示省略提示

GrepTool —— 文本搜索工具

作用

安全标记

限制

execute

- 提取并验证搜索模式
- 提取搜索路径（默认当前目录）
- 编译正则表达式（支持大小写敏感/不敏感）
- 如果目标是文件 → 直接搜索该文件
- 如果目标是目录 → 递归遍历，跳过 .git、target 等目录
- 对每个文件逐行匹配，输出匹配行（文件路径:行号:内容）
- 统计扫描文件数和匹配数

ListDirectoryTool —— 目录列表工具

作用

安全标记

execute

- 提取路径参数（默认当前目录）
- 检查路径是否存在且是目录
- 根据是否递归选择遍历方式
- 过滤掉自身、按名称排序、限制条目数
- 输出 [D] 标记目录、[F] 标记文件
- 递归模式显示相对路径，非递归模式只显示文件名

WriteFileTool —— 写文件工具

作用

安全标记

限制

execute

- 提取路径和内容参数
- 检查内容大小
- 规范化路径
- 检查目标不是目录
- 如果文件已存在且是二进制文件，拒绝覆盖
- 创建父目录（如不存在）
- 根据追加/覆盖模式选择写入方式
- 返回写入结果（字符数、路径、模式）

DeleteFileTool —— 删除文件工具

作用

安全标记

execute

- 提取路径参数
- 规范化路径
- 检查文件是否存在
- 检查是否是普通文件（不支持删除目录）
- 执行删除
- 返回删除结果

BashTool —— Shell 命令工具

作用

安全标记

安全限制

- 默认禁用，需用户手动开启
- 内置危险命令检测（拒绝 `rm -rf /`、`mkfs` 等命令）
- 默认超时 10 秒
- 输出超过 50,000 字符自动截断

execute

- 提取命令参数
- 检查命令是否为危险命令
- 提取超时参数
- 创建进程构建器（Linux 用 `bash`，Windows 用 `cmd`）
- 合并标准错误到标准输出
- 启动进程
- 逐行读取输出，超过限制则截断
- 等待进程结束（带超时），超时则强制终止

- 组装结果（命令 + 输出 + 退出码）
- 退出码为 0 返回成功，否则返回失败

核心流程：Agent Loop 伪代码

用户输入

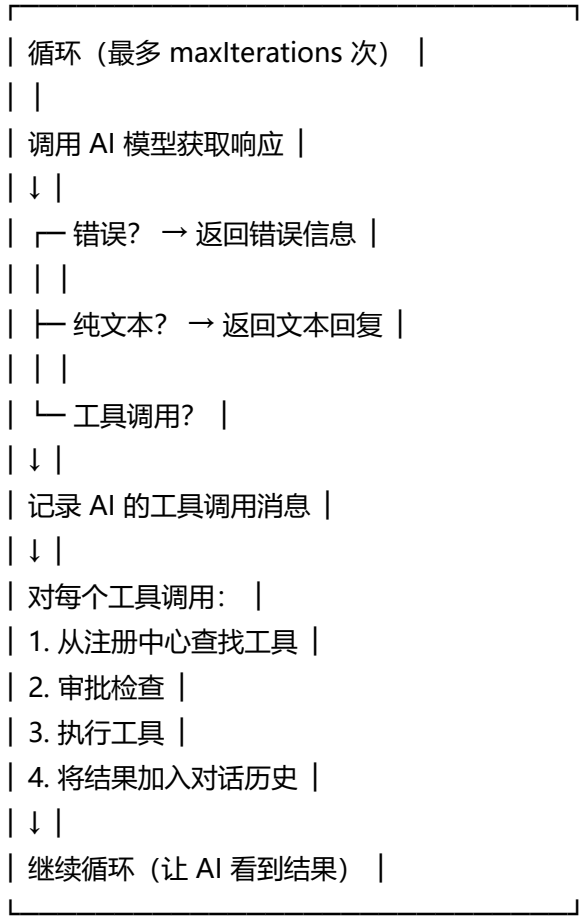
↓

将用户消息加入对话历史

↓

构建系统提示词（身份 + 工具列表）

↓



↓

超过最大次数 → 返回提示

审批流程伪代码

工具调用请求

↓

策略检查

└─ Bash 未启用？ → 拒绝

└─ 路径在工作区外？ → 拒绝

└─ 受保护路径？ → 拒绝

- └─ 删除根目录? → 拒绝
- └─ 只读工具? → 自动通过
- └─ 其他 → 需要审批

↓

检查审批缓存

- └─ 缓存命中 (之前批准过) → 自动通过
- └─ 缓存命中 (之前拒绝过) → 自动拒绝
- └─ 缓存未命中

↓

请求用户确认

- └─ 用户同意 → 批准, 存入缓存
- └─ 用户拒绝 → 拒绝, 存入缓存
- └─ 用户取消 → 拒绝, 不缓存